

CANDY DIALOGUE ENGINE

Transitions

1.1.0

Transitions let dialogues move from one Block to another, either permanently or temporarily. They are used to navigate narrative branches, conditional paths, or structural shortcuts within a conversation.

Candy DE supports four transition commands: §Jump, §Bridge, §Return, and §End. A fifth command, §LM, acts as a static marker used by the others.

To properly use transitions, it's important to understand what happens when a Block ends:

Normally, since there are no more lines to process, the dialogue would end. However, if a Block was entered through a §Bridge command, the engine would instead return to the previous Block and continue from the line after the §Bridge command.

§Jump

§Jump performs a **permanent transition** to another Block.

Once executed, the dialogue continues from the target Block and treats it as the new top level.

This means that any earlier Blocks are forgotten (including previous bridges between Blocks, or pending choice menus or condition chains). The transition effectively resets the dialogue flow to a clean state, as if the new Block were the starting Block of the dialogue.

In other words, use §Jump when you want to move the dialogue forward permanently.

The §Jump command can be provided the name of a Conversation and the name of a Block, as well as a line index or an §LM reference. If you don't provide a Conversation or a Block, the engine will default to the same Conversation or Block that the §Jump command is located in.

For lines, a number will be treated as a line index, while any text (including text containing numbers) will be treated as an §LM reference. If no line data is provided, the engine will default to the first line of the indicated Block (which is probably what you'll want to do most of the time).

Providing no data will cause the engine to §Jump to line 0 of the same Conversation and Block, repeating the same Block and potentially looping through it indefinitely.

§Bridge

§Bridge performs a **temporary detour** to another Block.

Unlike §Jump, it preserves earlier Blocks, creating *nesting*, and allows returning to the previous Block automatically when the branch ends.

In other words, §Bridge creates a detour — the dialogue leaves the current Block, runs through another one, and comes back when finished.

All previous nested choice menus and condition chains are preserved, passively waiting for the engine to return to them.

Therefore, use §Bridge when you want to create a temporary detour before returning to the previous Block and continuing where you left off.

§Bridge can be provided the same data as §Jump: references to a Conversation, a Block, and a line.

§LM

§LM (short for Line Mark or Line Marker) is a special line that serves as a static marker within a Block. It has no effect on dialogue flow: the engine just skips it and continues to the next line.

The purpose of §LM is to let §Jump or §Bridge reach specific lines inside a Block by name rather than by line index.

Line indexes are fragile: they'll change if you edit the Block and add or remove lines. However, they are simple to use when you are certain a line's index won't change: just provide the number to a §Jump or §Bridge command.

§LM commands, on the other hand, require inserting them ahead of time in a Block. But the upside is that even if line indexes shift, §LM references continue to work, reliably guiding §Jump and §Bridge to themselves.

To use an §LM command, you must write a reference in their data. These references can be anything, except a string made only of numbers as the engine will interpret this as a line index.

§LM commands inside a same Block must not have the same reference, otherwise the engine will always transition to the first one. However, §LM commands located in different Blocks can share the same references safely.

§End & §Return

Both of these commands are straightforward: §End ends the dialogue instantly, no questions asked.

§Return, on the other hand, returns to the previous Block, but only if the §Bridge command was used to reach the current Block. Otherwise, §Return will end the dialogue as well.

Normally, dialogues automatically return or end at the end of a Block. Therefore, §End and §Return would only be used when you want to exit a Block early.

Importantly, you don't need to end each block with §End or §Return.

Use Cases

In practice, you would use transitions almost exclusively in three situations:

- 1) At the end of a Block, to connect to another Block.
- 2) For conditional branching midway through a Block.
- 3) With choices, where a choice transitions to another Block.

Conditions and choices don't actually require the `$Jump` or `$Bridge` commands: they have built-in Jump or Bridge transitions for simplicity (see the [Conditions](#) and the [Player Choices](#) guides for details). Therefore, you would mostly only insert the actual `$Jump` and `$Bridge` commands at the end of Blocks.

One particular scenario, however, is when you want to split a Block in multiple independent sections. Basically, creating 'mini Blocks' inside an actual Block:

Use `$LM` commands to mark the start of each section, and close each section with either `$Return` or `$End` commands as desired.

This can be convenient for grouping the different results of several conditions or choices in one Block, instead of littering your dialogue script with many Blocks. It also makes referencing them easier, as you'll provide the same transition Conversation and Block name for each condition and choice, only changing the line reference each time.